

The random approximate values over algebraic types

Alexander Towell
atowell@siue.edu

Abstract

We define the semantics of *random approximate values* over algebraic data types.

Contents

1	Introduction	1
2	Primitive approximate <i>values</i>	1
2.1	Void type	1
2.2	Unit type	2
2.3	Sum types	2
2.4	Product types	2
2.5	Exponential types (functions)	2
3	C++ implementation	3
3.1	Approximate value monad	5
3.2	Type-erasure	5

1 Introduction

The primary mechanism by which a value is an *approximation* is given by the random approximate map model. If a first-order random approximate map of type $X \xrightarrow[\omega]{\epsilon} Y$ takes in an exact value X then it maps to a random approximate value y_ϵ^ω of type Y .

We denote that the distribution of values over a type X take on random approximations with $X^\pm$. The type is still the same, only the *values* are different with respect to some objective standard, e.g., if f maps a value a to b , then ...

A *type* is a set and the elements of the set are called the *values* of the type.

These *values* are approximate values if, according to some objective function they should be x but take on a range of possible values according to the random approximate value model.

An *abstract data type* is a type and a set of operations on values of the type. For example, the *integer* abstract data type is defined by the set of integers and standard operations like addition and subtraction. A *data structure* is a particular way of organizing data and may implement one or more abstract data types.

2 Primitive approximate *values*

Here are a basic list of primitive algebraic data types and operations on types.

These are *first-order* approximate types since t

2.1 Void type

The most primitive type is the *empty set* type, denoted by `Void`. There are no elements in the empty set and therefore it is not possible to construct values of this type. There is only one function in the set $\text{Void} \mapsto X$, known as the *absurd* function since it can never be invoked. Since `Void` has no values, $\text{Void}^\pm$ is equivalent to `Void`.

$\text{Void}^\pm$ is necessary to complete the algebra of algebraic data types, but serves only a theoretical purpose.

2.2 Unit type

The *unit* type, denoted by `Unit`, is isomorphic to any set with only a single element, i.e., any *singleton set*. The set $\text{Unit} \mapsto X$ has a cardinality $|X|$ and the set $X \mapsto \text{Unit}$ has a cardinality 1. If we are talking about *partial* functions, then $X \rightarrow \text{Unit}$ has cardinality 2^X .

The set $\text{Unit} \mapsto \text{Unit}$ has a cardinality of 1, which is the identity function $\text{id}: \text{Unit} \mapsto \text{Unit}$.

The approximate unit type $\text{Unit}^\pm$ is necessary to complete the algebra, but given a void `Unit` type, similar to the `Void` type there is no uncertainty about its value, i.e., $\text{Unit}^\pm$ is equivalent to `Unit`.

In addition, if there is some function $f: \text{Unit} \mapsto X$ its approximate analog is $f^\pm: \text{Unit} \mapsto X$, which models an *approximate* constant.

2.3 Sum types

A sum type $X + Y$ is the disjoint union of types X and Y . The *first-order* approximate sum type $(X + Y)^\pm$ is an approximate of the type $X + Y$.

The *higher-order* sum type is different, e.g., $X^\pm + Y^\pm$ is a higher order sum type, as is $X^\pm + Y$ and $(X^\pm + Y^\pm)^\pm$ is even a higher order.

If we replace X and Y by $X^\pm$ and $Y^\pm$, we have a sum type $X^\pm + Y^\pm$.

Values of these types are naturally constructed from approximate maps that map to the type $X + Y$.

2.4 Product types

A product type $X \times Y$ is the Cartesian product of types X and Y . If we replace X and Y by $X^\pm$ and $Y^\pm$, we have a product type $X^\pm \times Y^\pm$.

Values of these types are naturally constructed from approximate maps that map to the type $X + Y$ and form a random approximate set over the type $X \times Y$.

2.5 Exponential types (functions)

These have already been discussed. Random approximate maps are the same thing as random approximate exponential types.

When we generate a set of approximate value types and a set of approximate functions over those value types and compose them together to generate a *program*, we may consider this composition to be an *approximate* program.

TODO: A *partial function* is not defined on entire domain. We allow elements not in the preimage, i.e., not defined by the partial function, to either correctly map to *nothing* or, map to

some other element in the codomain. The *false mapping rate* ε_y for element y may be specified explicitly, i.e.,

$$P[f(x) = y \mid x \notin \text{dom}(f)] = \varepsilon_y, \quad (1)$$

and the *total* false mapping rate is

$$\sum_{y \in \text{codom}(f)} P[f(x) = y \mid x \notin \text{dom}(f)] = \varepsilon, \quad (2)$$

by the fact that each outcome is mutually exclusive (a function f maps to only one element). Alternatively,

$$\sum_{y \in \text{codom}(f)} P[f(x) \neq y \mid x \notin \text{dom}(f)] = 1 - \varepsilon = \eta. \quad (3)$$

Example 1 Suppose we have a predicate function $f: X \mapsto \text{Bool}$, i.e., set indicator. Then, we may construct a partial function from f with an approximate map over those elements that return **true**. If we specify a false mapping rate $\varepsilon_{\text{false}}$, this predicate is isomorphic to an approximate set over X with a false positive rate $\varepsilon = \varepsilon_{\text{false}}$.

An optimal approximate map obtains the information-theoretic lower-bound on the expected space complexity, $-1.44 \log_2 \varepsilon$ bits per positive element.

While a random approximate set is in practice simpler to implement, theoretically an optimal approximate map is both fully generalized (for any function over any domain and codomain) and obtains the same space efficiency.

The obfuscated approximate map has the same characteristics, and the optimal implementation is a black box that is able to increase obfuscation power for less space efficiency. *WORK THIS OUT EXACTly*, and move all of this out of the example env of course.

Remark. NOTE: Clarify: we call a value an approximate value when it should be x but there is a probability it is something else due to the approximate map or some other noisy process. $\triangle$

Most approximate maps are black boxes. They introduce an approximation error, and in addition, the inputs may also have an approximation error. At that point, the values being mapped to are higher-order random approximate values.

3 C++ implementation

In programming languages, *composite* types are typically composed in two ways, the *sum type* and the *product type*. The product type is a Cartesian product of types (sets).

For instance, a product type may be the Cartesian product of integer and Boolean types, $\mathbb{Z} \times \{0, 1\}$. In the C++ family of programming language, this product type may be represented by `pair<int, bool>`.

Since it may be inconvenient to refer to member types by their respective tuple indices, programming languages typically allow the indices to be *labeled*. In C++, keywords like *struct* and *class* are commonly used to provide named product types.

A data type may be thought of as a product type with *invariants*, or *constraints* on which values member types may be assigned. Thus, it may be thought of as a *relation* or *correspondence* over the product type. In many cases, this may make approximate relations unsuitable for representing data

types, e.g., a tuple that violates one or more of the invariants may be generated from an objective value of the data type.

However, if an approximate relation does violate the invariants, it can simply be considered invalid. In this case, we may pose questions like, what is the probability that an approximate data type generates an invalid result?

Viewing a *type* as a set, most programming languages have primitive types like *integers*, *Booleans*, and *characters*. In C++, these are respectively denoted by `int`, `bool`, and `char` with cardinalities given respectively by 2^{32} , 2^8 , and 2^1 . Any type needs one or more *value constructors* to construct objects that model values in that type. For instance, in C++ the value that denotes the Boolean value of *true* is constructed with the syntax `bool{true}`.

The *unit* type is special singleton set with a single value, i.e., a cardinality of 2^0 . In C++, the confusing notation of `bool{}` denotes the unit type (and the single value).

Remark. *The absurd type is a special type with zero values, i.e., the empty set. Since there are no values in the absurd type, no values of this type can be constructed. There is no primitive absurd type in C++.* $\triangle$

A *product type* is the n -fold Cartesian product of zero or more types where the zero-th product is the unit type. For instance, in C++, `std::pair<int, bool>` is a *named* product type and `std::tuple<int, bool>` is the *unnamed* counterpart, where both are product types $\text{int} \times \text{bool}$. The cardinality of this product type is $2^8 \cdot 2^1 = 512$. One way a particular value of this product type may be constructed is `std::pair{1, true}`.

The values of a sum type are typically grouped into several classes, called variants. The set of all possible values of a sum type is the disjoint union of the sets of all possible values of its variants. For instance, in C++, `std::variant<int, bool>` is the sum type $\text{int} + \text{bool}$, which has a cardinality of $2^8 + 2^1 = 258$. One way a particular value of this sum type may be constructed is `std::variant{1, true}`. A particularly useful type in C++ is `std::optional<T>`, which conceptually models the sum type $T + \text{unit}$ where `unit` denotes the variant “not a value of type T ”.

Remark. *This is not valid C++ syntax, even though the unit type value should be first-class.* $\triangle$

The cardinality of the `std::optional<T>` is the cardinality of `T` plus 1. We label the value in this singleton *nothing* and may test whether a particular value is either *nothing* or alternatively a value in `T`.

Exponential types are *functions*. In C++, `std::function<T>` is the set of functions $X \mapsto T$, which has a cardinality of $2^{|X|}$. Usually, a more convenient syntax is used, like `std::function<T> f`. The constant function of the exponential type $X \mapsto T$ may be constructed with the definition `std::function<T> f{[]{}{T{}}}`.

Remark. *The exponential type is of little practical value and, in C++, usually denotes a procedure that causes side-effects like writing to IO.* $\triangle$

Recall that any subset of a set corresponds to a *relation*. Types are *subsets* of the algebraic types where subsets are defined by *invariants*. We may compose primitive types to specify a variety of *compound* types.

Example 2 *Rationals may be implemented as a product type of two integers, where the first and last elements of the tuple represent the numerator and denominator respectively. If the invariant is that the denominator is not 0, then a value constructor $\text{rational} : \text{int} \times \text{int} \mapsto \text{rational}$ that takes a numerator and denominator and outputs either a rational or, if the invariants are violated, nothing, is given by `std::optional{rational(n, d)}`.¹*

¹Alternatively, the value constructor can be a *partial function* $\text{rational} : \text{int} \times \text{int} \dashrightarrow \text{rational}$ that is undefined for input $\langle x, 0 \rangle$ for any $x \in \text{int}$.

The *expected operators on rationals*, like *addition*, may be implemented so that *models the concept of rationals*.

The *type* is a subset of the *product type* and is thus a binary relation on $\mathbb{Z} \times \mathbb{Z}$.

Remark. We implemented *as a product type* and a *value constructor* for pedagogical reasons, but generally programming structures like *class* are utilized since they facilitate important concepts like *encapsulation*. $\triangle$

Each of these types and operators has a corresponding random approximation, e.g., the *exponential type* is just a random approximate map, and the *relations* that define types are just random approximate relations with *deterministic* properties that model the invariants.

The invariants may not be easy to satisfy, and so a random approximation relation of the corresponding type may not be practical. However, when the invariants can be satisfied, we may implement *random approximate algebraic data types* of the *algebraic data types*, e.g., we can compose random approximate algebraic data types as before to construct compound random approximate data types of the corresponding compound algebraic data type.

This may not seem particularly useful, but it permits space-efficient representations and, moreover, concepts like *oblivious algebraic data types* may be based on it with some notion of closure.

3.1 Approximate value monad

Talk about lifting functions.

Suppose we have a function $f: X \mapsto Y$.

and we apply f to an approximate value of type X , denoted by $a(X)$,

$f(a(X)): X \xrightarrow[\omega]{\varepsilon} Y$.

The approximate value parameterized by X , denoted by $a(X)$, is a monad type similar to the *maybe* monad discussed previously, except significantly more complicated if *fully* implemented.²

The general type of the approximate value monad is simply defined as which has a computational basis given by the following overload set.

Given an approximate value of type X , its *false positive rate* is given by its *false negative rate* is given by its value is given by and its conditional probability mass function is given by

Note that the false positive and false negative rates are the result of using the distance function $d(a, a) = 0$ and otherwise $d(a, b) = 1$, $a \neq b$, to calculate the *expected* loss given respectively the negative (where d evaluates to 1 with respect to some ground truth) and positive (where d evaluates to 0) subsets of X .

If we give it a *loss* function, we may estimate its loss.

For particular value types, we specialize this template. which has a computational basis given by

The function $:(\mapsto) \times \mapsto$ is defined as

If we compose functions to construct an *algorithm* (or program), then if some of the values (e.g., functions or Boolean values) are replaced by approximate values, the algorithm becomes (in general) an approximate algorithm and we may deduce its false positive and negative rates as we did previously through, say, function composition.

If the algorithm includes *branching*, the algorithm is still an approximate algorithm, but in order to infer the false positive and false negative rate, we must go down *all* branches, which is in general not be tractable. However, we could estimate the result using *Monte-carlo* simulation.

²We may choose the trivial implementation that just tags it as an approximate value and let that bit of information carry through.

The approximate Boolean value is straightforward since Boolean types are the natural predicate return type. More importantly, there are only two values of this type.

3.2 Type-erasure

The fact that we are dealing with specific approximate types may be erased into some approximate type abstract data type that hides the concrete data type. It may be further erased to just a type, e.g., f_ε^ω may be erased to $f^\pm$ which may be erased to f . TODO: convert to types not particular elements in the type